Neural Networks Optimization

HW/SW Codesign - Week 11

Today's Topics

- Neural network architectures review
- Optimization objectives
- Quantization
  - Uniform quantization
  - Quantization options
  - Weight and activation quantization
  - Quantization-aware training
  - Efficiency gains
- Pruning and sparsity
  - Unstructured vs structured pruning
  - The lottery ticket hypothesis
- Neural architecture search
  - Search space
  - Objective estimation
  - Constraints
  - Multi-objective optimization
  - AutoML
- Entire design space

Optimization Objectives

What we want to achieve

Optimization Objectives

- Accuracy
- Efficiency:
  - Latency
  - Energy/Power
  - Size (flash)
  - Memory (RAM)
- Inference efficiency only (training efficiency outside scope of course)

Accuracy-Efficiency Trade-Off

- Larger model
  - ⇒ Better accuracy
  - ⇒ Worse latency, size, memory, energy
- Optimization
  - ⇒ Better latency, size, memory, energy
  - ⇒ Sometimes worse accuracy
- The task:
  - Find optimizations with minimal accuracy impairment
  - Balance accuracy vs efficiency

Optimization Dimensions

- Model representation:
  - Change the computations
  - Techniques:
    - Pruning
    - Neural Architecture Search (NAS)
- Numeric precision:
  - Change the numeric precision of the computations
  - Technique: Quantization
- Architectural efficiency:
  - Change the implementation of the computations
  - Techniques:
    - Operator fusion
    - Hardware mapping

Quantization

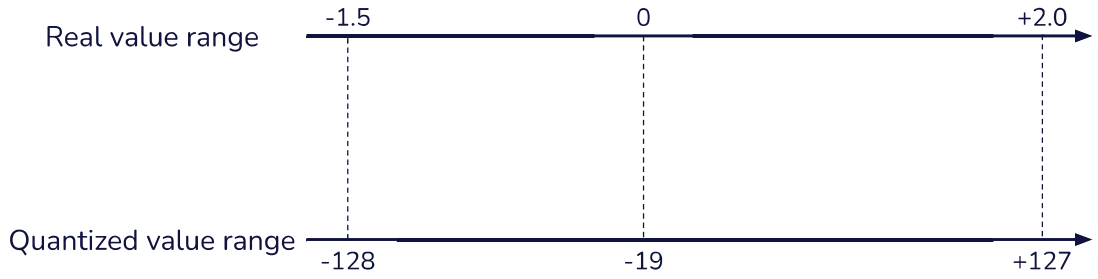
How to speed up a model by reducing precision

Quantization

- Lower numeric precision
  - Example: 32-bit floating point ⇒ 8-bit integers
- Reduces latency, energy, size and RAM
- Often small accuracy degradation
  - Quantization errors are a kind of noise
  - Machine learning models are generally insensitive to noise

Uniform Quantization

- Conversion is a linear or affine mapping
  - Full real value range ↔ Full quantized value range
- Example - Weights of a layer range from -1.5 to +2.0 to 8-bit integers:



Uniform Quantization

- Given:
  - Range of real values r_{min}, r_{max} (e.g. -1.5 ... +2.0)
  - Range of quantized values q_{min}, q_{max} (e.g. -128 ... 127)
- Determine **scale** *s* and **zero point** *z* such that:
  - $r_{min} = s \cdot (q_{min} - z)$
  - $r_{max} = s \cdot (q_{max} - z)$
- Quantization of a value *r*:
  - $q = \text{clip}_{q_{min},q_{max}}(\text{round}(\frac{r}{s} + z))$
- Dequantization of a value *q*:
  - $r = s \cdot (q - z)$

Quantization Options

- Precision:
  - Floating point: FP16, BF16, FP8, etc
  - Integers: INT16, INT8, INT4 etc
- Target:
  - Weights only
  - Full quantization (weights and activations)
- Granularity:
  - Layer-wise (also called per-tensor)
  - Channel-wise (also called per-axis)
- Symmetry:
  - Symmetric: Zero point $z = 0$
  - Asymmetric: Zero point $z \neq 0$

Precision: Floating-Point Formats

- FP32: 
  - 1 sign, 8 exponent, 23 fraction
- FP16: 
  - 1 sign, 5 exponent, 10 fraction
- BF16: 
  - 1 sign, 8 exponent, 7 fraction (easier to convert from FP32)
- FP8 (E4M3): 
  - 1 sign, 4 exponent, 3 fraction

Precision: Integer Formats

- INT16
- INT8
- INT4
- Binary
  - 1-bit: 0, 1
- Ternary
  - 2- (or 1.58-)bit: -1, 0, 1

Target: Weights-Only Quantization

- Only weights are quantized
- Activations remain in full precision
- Arithmetic:
  - Full precision: Weights are dequantized before multiplication
  - Quantized: Activations are quantized and dequantized dynamically
- Efficiency gains:
  - Model size: Reduced (e.g by 4x)
  - RAM demand: Same as full precision
  - Latency/energy: Depends

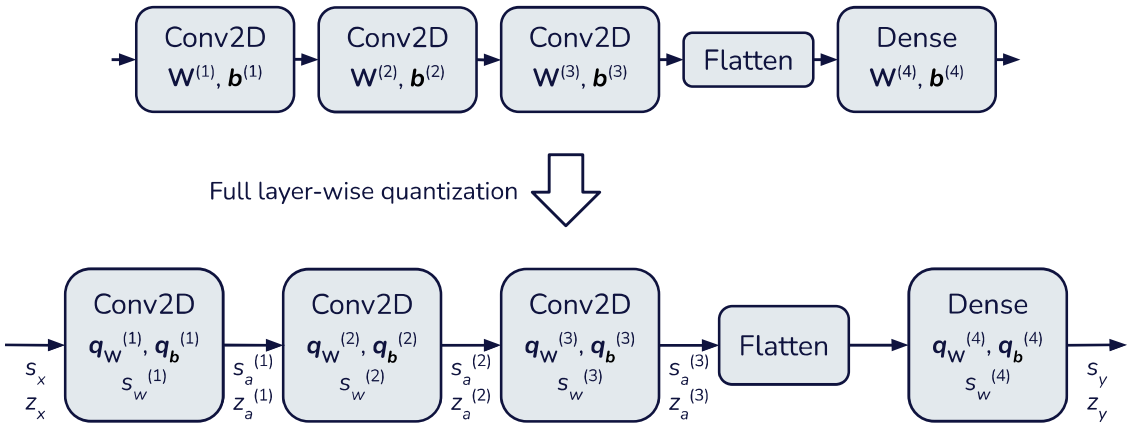
Target: Full Quantization

- Weights and activations are all quantized
- Arithmetic: Quantized with extra bit width (e.g. INT32)
- Efficiency gains:
  - Model size: Reduced (e.g. by 4x)
  - RAM demand: Reduced (e.g. by 4x)
  - Latency/energy: Reduced
- Complications:
  - Different scales and zero points for weights and activations
  - Activation scale and zero point requires **calibration**

Granularity: Layer-Wise vs Channel-Wise Quantization

- Layer-wise quantization:
  - Scale and zero point stored per layer
  - Example: 2D convolution weights ⇒ r_{min} and r_{max} over entire 4D weight matrix
- Channel-wise quantization:
  - Scale and zero point stored per filter/channel
  - Example: 2D convolution weights ⇒ r_{min} and r_{max} over each 3D filter

Model Quantization



Model Quantization

Full model quantization computes the following for every layer:

- s_w: Scale of weights q_W
  - If channel-wise, one for every channel ($s_{w,0}$, $s_{w,1}$, ...)
  - Typically no zero point for weights due to symmetry around 0
- q_w: Quantized weights
- q_b: Quantized bias
- s_a: Scale of activation q_a
- z_a: Zero point of activation q_a

Model Quantization: Weight and Bias Quantization

For every layer (or channel):

1. Max absolute weight is found
2. Weight scale is computed:
   - $s_w = \frac{\max(\text{abs}(W))}{q_{max}}$
3. Quantized weights and biases are computed:
   - $q_W = \text{clip}(\text{round}(\frac{W}{s_w}))$
   - $q_b = \text{clip}(\text{round}(\frac{b}{s_w s_x}))$, where s_x is the activation scale from the previous layer

Model Quantization: Activation Quantization

1. Engineer provides a **representative dataset**
   - Typically a subset of the inputs of the training set (e.g. x_train[0:1000])
2. **Calibration** runs prediction on the representative dataset
   - Min and max activation values (r_{min} and r_{max}) are registered for every layer
3. For every layer, activation scale and zero point are computed:
   - $s_a = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$
   - $z_a = q_{min} - \frac{r_{min}}{s_a}$

Inference with Quantized Model

- q_x = Input or activation from previous layer
  - scale = s_x, zero point = z_x
- $q'_a = \phi(\sum q_w(q_x - z_x) + q_b)$
  - E.g. INT32
  - scale = $s_w \cdot s_x$, zero point = 0
- $q_a = \text{clip}(\frac{s_w s_x}{s_a} q'_a + z_a)$
  - E.g. INT32 arithmetic, then converted to e.g. INT8
  - scale = s_a, zero point = z_a

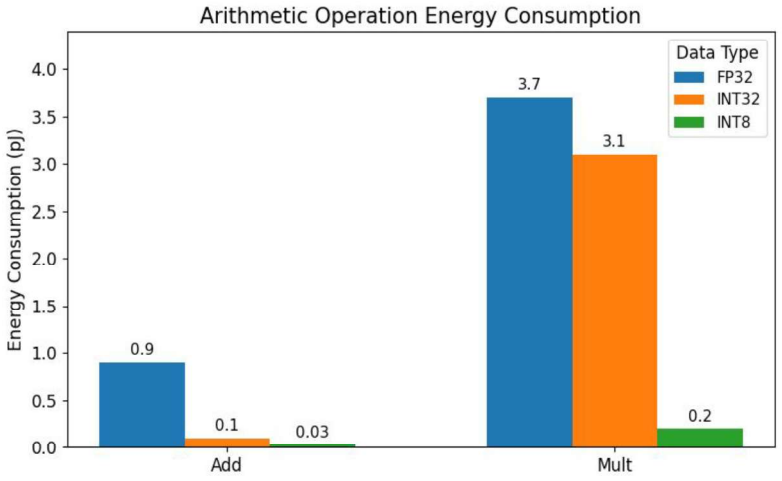
Post-Training Quantization vs Quantization-Aware Training

- Post-training quantization (PTQ):
  - Quantization after training
  - ⇒ Simpler, but may degrade accuracy more
- Quantization-aware training (QAT):
  - Make the model robust to quantization noise during training
  - ⇒ Better accuracy

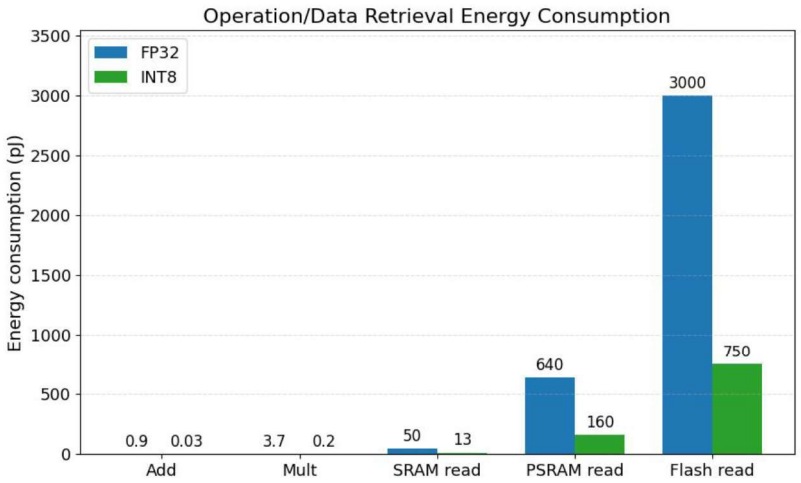
Quantization-Aware Training (QAT)

- Weights and activations are quantized during forward pass
- Full precision in the backward pass
- Observer module
  - Collects statistics
  - Used to calibrate quantization
- No drawback except more complicated training

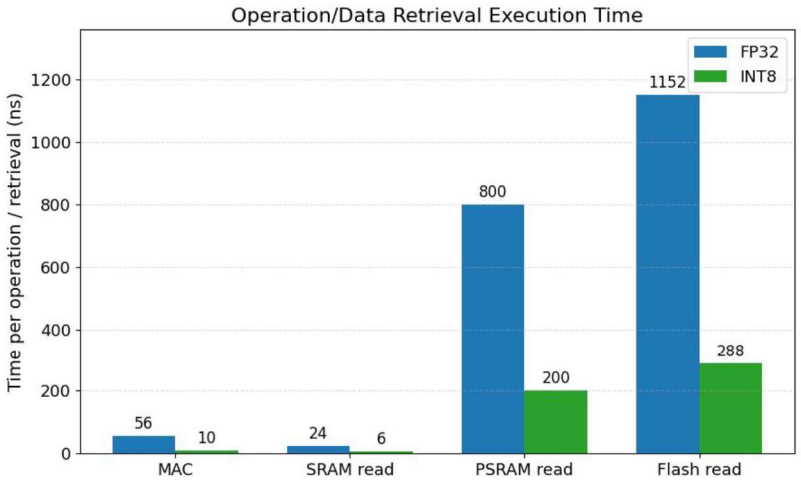
Efficiency Gains: Energy - Arithmetic Operations



Efficiency Gains: Energy - Data Read



Efficiency Gains: Latency



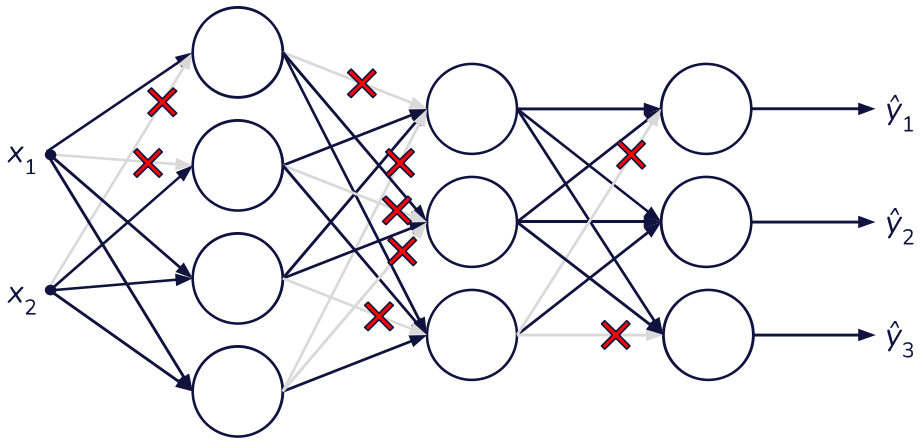
Accuracy Penalty

- Accuracy penalty depends on model and data
- Always evaluate the quantized model and compare with the full precision model
- If accuracy drops:
  - Check for quantization-sensitive activation functions, e.g. Swish in MobileNetV3
  - Consider quantization-aware training (QAT)
  - Consider if quantization is needed for the application

Pruning and Sparsity

How to remove unnecessary computations

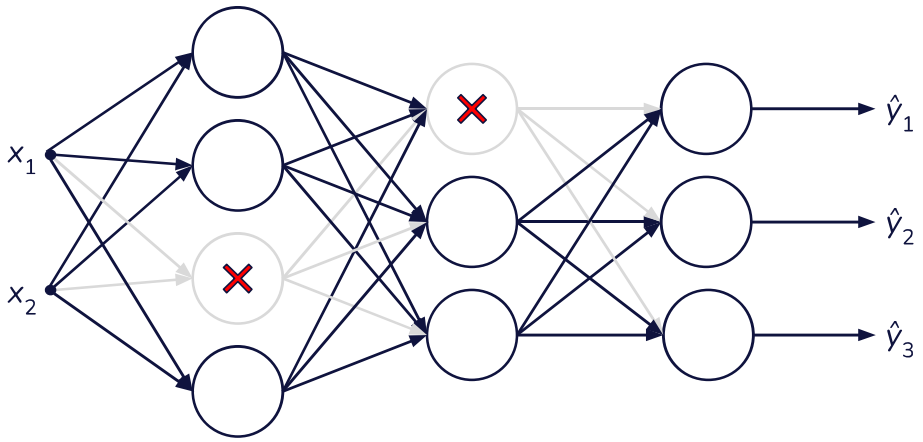
Pruning



Unstructured Pruning

- Also known as **weight pruning**
- Sets certain weights to 0
- A pruned model is known as **sparse** (as opposed to dense)
- Little impact on accuracy
- Problem with exploiting for efficiency:
  - The weight matrix W remains the same size
  - Inference libraries and hardware are optimized for matrix multiplication
  - ⇒ Exploiting unstructured sparsity requires specialized libraries and hardware

Structured Pruning



Structured Pruning

- Prune entire neurons (fully connected) or filters (convolution)
- Weight matrix dimensions are reduced
  - ⇒ Easy to exploit
  - ⇒ Reduced model size, RAM, latency and energy
- Stronger impact on accuracy

Pruning Algorithm

1. Select weights/neurons/filters to prune:
    - Magnitude-based pruning:
        - Determine a **pruning threshold** τ
        - Unstructured: Prune weight w where $|w| < \tau$
        - Structured: Prune sub-matrix $\boldsymbol{w}$ where $||\boldsymbol{w}|| < \tau$
2. Prune:
    - Unstructured: Set weights to 0
    - Structured: Remove neurons/filters from weight matrices and bias vectors
3. Fine tune on training set to compensate for removed elements
    - Unstructured: Freeze pruned weights to 0
4. Evaluate
5. Done (one-shot) or repeat (iterative)

Lottery Ticket Hypothesis

- Famous paper from 2018
- Refined iterative unstructured pruning:
    - Initialize dense network randomly; remember initial weight values
    - Train as normal
    - Repeat:
        i. Select weights and prune
        ii. Restore remaining weights to their initial values
        iii. Train from scratch with pruned weights frozen to 0
- Results:
    - Pruned more than 90% of weights
    - Improved accuracy after pruning

Neural Architecture Search

How to automate neural network design

Neural Architecture Search

- Automated search for efficient neural network architectures
- Define a design space
  - Layers, layer configurations, connections, etc
- Algorithm:
  - Generate a candidate model architecture
  - Train and evaluate
  - Repeat
- Differences from hyperparameter tuning:
  - Explores more architecture details, for example:
    - Layer types
    - Skip connections
  - Joint search for accuracy and resource efficiency

Neural Architecture Search: Setup

- Design space:
  - Layer types and counts
  - Layer configurations, for example:
    - Number of neurons/filters
    - Filter size
    - Activation function
  - Topology (how layers are connected)
- Objectives/constraints:
  - Accuracy or other performance metric (precision, recall, F1, etc)
  - Efficiency (latency/energy/RAM/flash)
- Search algorithm:
  - Evolutionary
  - Reinforcement learning models
  - Gradient descent

Objective Estimation

- Accuracy
  - Train and measure accuracy on validation set
- Compute (e.g. number of MACs), model size
  - Can be calculated precisely
- Peak RAM
  - Proxy: Max input+output tensor sizes over layers
- Latency, energy consumption
  - Proxy: Compute
    - Unreliable; depends on operation hardware implementation
  - Hardware in the loop
  - Prediction model

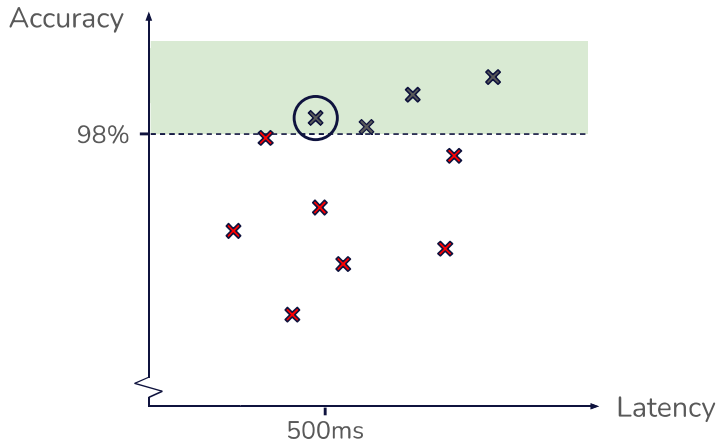
Objectives vs Constraints

For each metric, determine whether it's an objective or a constraint:

- Optimize for efficiency under accuracy constraint
- Optimize for accuracy under efficiency constraint
- Unconstrained multi-objective optimization
  - ⇒ Multiple solutions

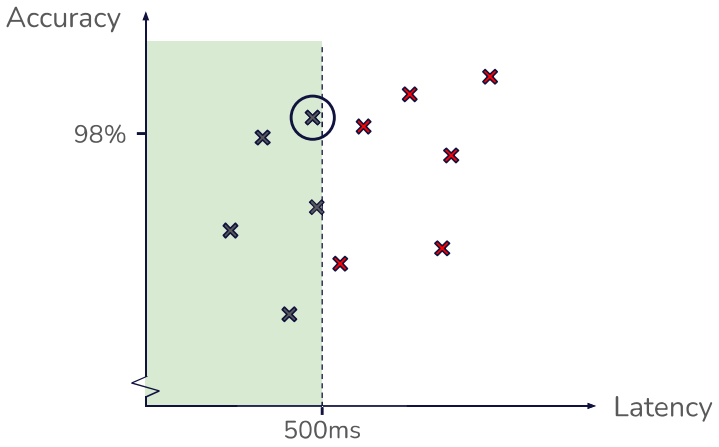
Objectives vs Constraints

Optimize for latency with constrained accuracy:



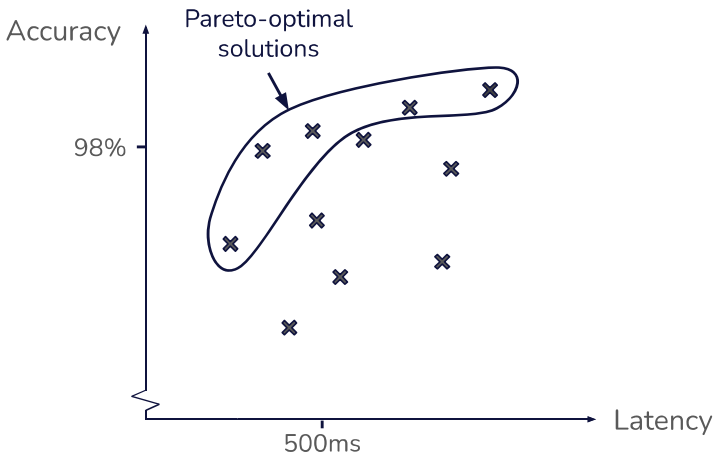
Objectives vs Constraints

Optimize for accuracy with constrained latency:



Objectives vs Constraints

Unconstrained multi-objective optimization:



NAS Setup Example: Keyword Spotter

- Design space:
  - Number of convolution layers: [2, 3, 4]
  - Number of depthwise separable convolution layers: [2, 3, 4]
  - Number of fully connected layers: [1, 2, 3]
  - Layer size: [16, 24, 32, 40, 48, 56, 64] per layer
  - Filter widths: [3, 4, 5, 6, 7] per convolutional layer
  - Skip connections: [false, true]
  - Batch normalization: [false, true]
- Objectives:
  - Maximize accuracy
  - Minimize latency
- Search algorithm:
  - NSGA-II

NAS in Research: EfficientNet

- ImageNet top-scorer 2019
- Used NAS to find optimal solutions
- Published 8 variants on the pareto front

NAS in Research: EfficientNet

Variant	Layers	Parameters	Operations	Accuracy
B0	132	5.3M	0.39B	77.1%
B1	186	7.8M	0.70B	79.1%
B2	186	9.2M	1.0B	80.1%
B3	210	12M	1.8B	81.6%
B4	258	19M	4.2B	82.9%
B5	312	30M	9.9B	83.6%
B6	360	43M	19B	84.0%
B7	438	66M	37B	84.3%

AutoML

- Combines into one search pipeline:
  - NAS
  - Hyperparameter tuning
  - Model optimization
- Same techniques as NAS
- Active area of research

Entire Design Space

The knobs of an embedded ML problem

Entire Design Space

- Sensor settings:
  - Camera:
    - Resolution (pixels)
    - Color depth (mono/RGB)
    - Exposure and Gain
  - Microphone:
    - Sample rate (Hz)
    - Bit width (8/16/24)
    - Channels (mono/stereo/multiple)
    - Gain
  - Accelerometer:
    - Sample rate (Hz)
    - Range (2g/4g/8g/16g)

Entire Design Space

- Preprocessing hyperparameters:
  - Data augmentation parameters:
    - Augmentation types
    - Number of variants per original example
    - Amounts (gain, rotation, etc)
  - Audio and time series preprocessing:
    - Audio frame size (e.g. 512 or 1024 samples)
    - Frequency band or filter bank selection
    - Sliding window width
    - Sliding window stride
  - Image preprocessing:
    - Input image size

Entire Design Space

- Model architecture (1/2):
  - Input size (given by preprocessor)
  - Number of trainable layers:
    - 2D convolution
    - Depthwise separable convolution
    - 1D convolution
    - Fully connected
  - Non-trainable layers:
    - Batch normalization
    - Pooling
  - Network topology
    - Layer order
    - Skip connections

Entire Design Space

- Model architecture (2/2):
  - Layer configurations:
    - Number of neurons or filters
    - Activation function
    - Filter sizes
    - Padding: Valid or Same
    - Strides
    - Pool size

Entire Design Space

- Training hyperparameters:
  - Learning rate
  - Batch size
  - Number of epochs
  - Optimizer (SGD, Adam, etc)
  - Weight decay
  - Early stopping patience
  - For every trainable layer:
    - L1 regularization
    - L2 regularization
    - Dropout rate

Entire Design Space

- Optimization options:
  - Quantization:
    - Precision (FP16, FP8, INT16, INT8, etc)
    - Target (weights only or weights+activations)
    - Granularity (layer-wise or channel-wise)
  - Pruning
    - Magnitude threshold
    - Iterations

Other Optimization Opportunities

- System level optimizations:
  - Networking
  - Preprocessing
- Hardware-aware model optimization:
  - Design and optimize the model with specific hardware in mind
  - Examples:
    - FPU available ⇒ Quantize to FP16 or FP8 instead of INT8
    - Optimized kernels for depthwise convolution ⇒ Include depthwise convolution
    - 512 kB SRAM ⇒ Keep peak memory below 512 kB
- Hardware acceleration